

# Low-Level Design (LLD)

## 1. Module-Level Design

ingestion.py → Document processing + vector store creation  
nodes.py → LangGraph nodes  
graph.py → StateGraph definition + compilation  
app.py → CLI interface  
utils.py → Helper functions

---

## 2. Data Structures

```
class GraphState(TypedDict):  
    question: str  
    context: List[str]  
    answer: str  
    escalate: bool  
    intent: str # "technical" or "account"  
    messages: Annotated[list, "add_messages"]
```

### Core Objects

#### Document

**question** (*str*)

The current user input/query.

**context** (*List[str]*)

The text snippets retrieved from the PDF manual.

**answer** (*str*)

The final response generated by the AI (*or the human fallback*).

**escalate** (*bool*)

A flag that tells Streamlit whether to show the **Human Agent Panel**.

**intent** (*str*)

Internal routing flag used to classify queries (e.g. *technical* or *account-related*).

**messages** (*List*)

Full chat history managed by LangGraph's **add\_messages**.

## Chunk Structure

```
{
  "page_content": "Q1. How do I reset my router? \nPress and hold the Reset button at the back of the router for 10 seconds until all lights flash, and then reconfigure using the default SSID: TechNova_X1000.",

  "metadata": {
    "source": "customer_support_manual.pdf",
    "page": 0,
    "chunk_id": "c1a2b3c4..." # (Generated by ChromaDB)
  },

  "type": "Document"
}
```

## Embedding

384-dimensional vector (MiniLM)

---

# 3. Workflow Design (LangGraph)

## Nodes

### A. **classify\_intent** (The Gatekeeper)

- **Action:** Analyzes the user's question.
- **Logic:** Categorizes the query into:
  - **Technical** — e.g. *"reset router"*
  - **Account-related** — e.g. *"cancel subscription"*
- **Routing:** First conditional edge in the graph.
  - Technical queries → proceed to retrieval

- Account/financial queries → bypass retrieval and route directly to human escalation
- 

## B. **retrieve** (The Librarian)

- **Action:** Queries the ChromaDB vector store.
  - **Output:** Returns the **top 3** most semantically similar chunks from the PDF manual.
  - **Purpose:** Supplies relevant context for answer generation.
- 

## C. **grade\_documents** (Quality Control)

- **Action:** Performs a relevance check on retrieved documents.
- **Logic:** Prompts the LLM:

*“Does this document actually contain the answer to the user’s question?”*

- **Routing:** Conditional decision point:
    - If relevant → continue to answer generation
    - If not relevant → route to **human\_escalate**
  - **Purpose:** Prevents grounded but incorrect responses.
- 

## D. **generate** (The Writer)

- **Action:** Synthesizes the final response.
  - **Input:**
    - User question
    - Only documents graded as relevant
  - **Safety Measures:**
    - Uses grounded RAG context only
    - Includes post-processing to remove hallucinated tags like:
      - **[ /ASS ]**
      - other malformed model artifacts
- 

## E. **human\_escalate** (The Safety Net)

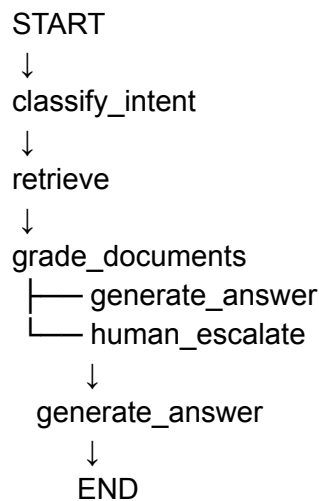
- **Action:** Prepares a handoff to a human agent.

- **Output:** Sets:

escalate = True

- **Effect:** Signals the Streamlit frontend to unlock the **Human Agent Panel**.
- **Triggered When:**
  - Account/financial queries
  - Low relevance retrieval
  - Low confidence responses
  - No supporting context found

## Graph Edges



## State

Shared state object passed across all nodes.

# 4. Conditional Routing Logic

## Answer Generation

**Implementation Method:** Relevant chunk found (*Binary “Yes” from Grader*)

- Query proceeds to response generation.

## Escalation (Low Quality)

**Implementation Method:** Irrelevant chunk found (*Binary “No” from Grader*)

- Routed to human escalation to avoid unreliable answers.

### Escalation (Empty Retrieval)

**Implementation Method:** Vector store returns **0 results**

- No supporting context available, so escalate.

### Escalation (Sensitive Queries)

**Implementation Method:** Intent classifier detects **Account/Refund** queries

- Bypasses retrieval and routes directly to a human agent.

### Clarification

**Implementation Method:** Not yet implemented

- Ambiguous queries currently escalate instead of triggering a clarification prompt.
- 

## 5. HITL (Human-in-the-Loop) Design

### Trigger Conditions

- **Irrelevant Context**  
Triggered when the `grade_documents` node returns "no".
  - **Complex Query**  
Triggered when `classify_intent` detects **Account/Refund** topics.
  - **Missing Context**  
Triggered when the vector store returns **zero results**.
- 

### Process Flow

- **Flag-Based Escalation**  
The graph completes its run and sets:

escalate = True

- **UI Intervention**  
Streamlit detects the escalation flag and unlocks a **Human Agent Panel**.
  - **History Injection**  
Human responses are appended to chat history, and the page reruns to refresh the UI.
-

## Persistence

- Uses **MemorySaver** to persist:
    - Conversation threads
    - Workflow state
    - Graph checkpoints in memory
- 

# 6. Interface Design

## Input Format

```
{  
  "question": "User's technical query here..."  
}
```

---

## Output Components

- **Answer**  
Generated response from either:
    - AI system
    - Human fallback
  - **Sources**  
Clickable expander in the UI showing retrieved document snippets.
  - **Escalation State**  
Visual warning/panel displayed when human assistance is required.
- 

## Interaction Mode

- **Streamlit Web App**  
Browser-based chat interface with:
    - Real-time feedback
    - Retrieval-backed responses
    - Human escalation support
-

# 7. Error Handling

## Retrieval Logic

- If no relevant chunks are found:  
→ Route to `human_escalate` node.
- 

## LLM Connection

- Uses `HuggingFaceEndpoint` connector.
  - Relies on API-side stability for model availability.
- 

## Vector Store

- Uses persistent local ChromaDB storage:

`./chroma_db`

---

## In-Process Validation

Nodes include debugging print statements to track:

- Intent classification
- Retrieval output
- Grading decisions

Used for developer-side monitoring and troubleshooting.

---

# Summary Architecture Flow

